

Reviewer Pack — Minimal Local System Rank in Affine Geometry and SAT Clause ...

Entry df31c2a1aaf1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 02:54:13 UTC

Minimal Local System Rank in Affine Geometry and SAT Clause Set Complexity

Entry ID: df31c2a1aaf1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 02:54:13 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): SAT Clause Sets

Statement:

For every k -SAT instance with m clauses, the minimal local system rank in affine geometry of the associated matroid is linearly correlated with the clause set complexity, such that $R_{\text{local}}(G) = \Theta(S_{\text{clauses}}(m))$, where $R_{\text{local}}(G)$ is the minimal local system rank of the matroid G derived from the clause set and $S_{\text{clauses}}(m)$ is the clause set complexity.

Rationale (proposer's reasoning):

Affine geometry has been applied in other fields to analyze the structure of combinatorial objects, such as graphs. The conjecture links this concept with SAT clause sets, which could reveal new structural insights into the hardness of SAT problems that are not immediately apparent through traditional methods.

Taxonomy category: AffineGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d1c999512d71adf7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal local system rank $R_{\text{local}}(G)$ and the clause set complexity $S_{\text{clauses}}(m)$ is ≥ 0.8 for all k -SAT instances with m clauses ($m \leq 40$). The conjecture is falsified if any seed produces a Pearson correlation coefficient < 0.8 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local System Rank in Affine Geometry AND SAT Clause Sets - Affine Geometry application in k-SAT clause set complexity - Correlation between $R_{\text{local}}(G)$ and $S_{\text{clauses}}(m)$ in matroids derived from SAT instances

Top relevant hits considered: - [http://arxiv.org/abs/1605.01678v2] The geometry of rank-one tensor completion - [http://arxiv.org/abs/0810.5165v2] Some computations about Kazhdan-Lusztig cells in affine Weyl groups of rank 2 - [http://arxiv.org/abs/0802.4323v1] Non-singular affine surfaces with self-maps - [http://arxiv.org/abs/2111.12700v2] Universality in long-distance geometry and quantum complexity - [http://arxiv.org/abs/2401.09234v2] SARRIGUREN: a polynomial-time complete algorithm for random k -SAT with relatively dense clauses - [http://arxiv.org/abs/2308.03822v1] Search for Eccentric Black Hole Coalescences during the Third Observing Run of LIGO and Virgo - [http://arxiv.org/abs/2509.08054v1] GW250114: testing Hawking's area law and the Kerr nature of black holes - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Find a row to swap with
            for j in range(i + 1, n):
                if A[j][i] != 0:
                    A[i], A[j] = A[j], A[i]
                    break
            else:
                raise ValueError("Matrix is singular")
            # Eliminate below the pivot
            for j in range(i + 1, n):
                factor = Fraction(A[j][i], A[i][i])
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    return A

def rank_of_matrix(A):
```

```

n = len(A)
r = 0
for i in range(n):
    if all(A[i][j] == 0 for j in range(r)):
        continue
    for j in range(r, n):
        A[i], A[j] = A[j], A[i]
        break
    r += 1
    for j in range(i + 1, n):
        factor = Fraction(A[j][i], A[i][i])
        for k in range(n):
            A[j][k] -= factor * A[i][k]
return r

def min_local_system_rank(clause_set):
    # Convert clause set to a matrix
    n = len(clause_set)
    m = max(len(c) for c in clause_set)
    A = [[0] * (m + 1) for _ in range(n)]
    for i, clause in enumerate(clause_set):
        for j, literal in enumerate(clause):
            if literal > 0:
                A[i][j] = 1
            else:
                A[i][j] = -1
    return rank_of_matrix(A)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random k-SAT instance with m clauses
    k = 3 # Example value for k
    m = random.randint(5, 40)
    clause_set = []
    literals = set(range(1, 2 * m + 1))
    for _ in range(m):
        clause = random.sample(literals, k)
        clause_set.append(clause)

    # Calculate the minimal local system rank
    r_local = min_local_system_rank(clause_set)

    # Measure the clause set complexity
    S_clauses_m = len(set(literal for clause in clause_set for literal in clause))

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": math.nan, # Placeholder for actual calculation
        "instances_tested": m,
        "n_max": m,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

# Compute mean/std of metric_value
if all(isinstance(r["metric_value"], (int, float)) and not math.isnan(r["metric_value"]) for r in results):
    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
else:
    mean, std = "N/A", "N/A"

# Compute fraction of seeds where conjecture_holds
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

# Print final result
if all(r["metric_value"] >= 0.8 for r in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(r["metric_value"] < 0.8 for r in results):
    first_failing_seed = next(i for i, r in enumerate(results) if r["metric_value"] < 0.8)
    print(f"RESULT: FALSIFIED counterexample='Pearson correlation coefficient < 0.8' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cld00363.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_cld00363.py", line 74, in run_trial
    clause = random.sample(literals, k)
              ~~~~~
  File "/usr/lib/python3.12/random.py", line 413, in sample
    raise TypeError("Population must be a sequence. ")
TypeError: Population must be a sequence. For dicts or sets, use sorted(d).

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the Pearson correlation coefficient could not be calculated to verify the conjecture. | next: Re-run the test without errors to calculate the Pearson correlation coefficient and verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13850
2	preregistration	ollama_remote	glm4:latest	0	0	20165
3	novelty	ollama_remote	glm4:latest	0	0	17056
4	novelty	ollama_remote	glm4:latest	0	0	12789
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16833
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20526
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29959
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12223
9	judge	ollama_remote	glm4:latest	0	0	8721

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 152122 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/df31c2a1aaf1.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/df31c2a1aaf1.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/df31c2a1aaf1.tar.gz (if generated)